

Agentic Incident Response

AIR

Architecture & Design Document

End-to-End System Design for Director & VP Review

Platform: AIR v1.0	Stack: Java Spring Boot · Spring AI · React · Python	Classification: Confidential
--------------------	--	------------------------------

↓ 60% MTTR Reduction	L3 Auto Incident Autonomy	24/7 AI Monitoring	\$2.4M Projected Savings/yr
---	--	---	--

American Express India · Enterprise Architecture · 2025

Table of Contents

1

Executive Summary

3

2

Business Case & Strategic Value

4

3

Problem Statement & Current State

5

4

AIR Product Vision & Scope

6

5

High-Level Architecture

7

5.1

Architecture Layers

7

5.2

Autonomy Levels – L1, L2, L3

8

6

Low-Level Design

9

6.1

Alert Ingestion & Normalisation

9

6.2

AI Orchestration Engine

9

6.3

Agent Execution Framework

10

6.4

MCP Tools & Integrations

10

7

API Specification

11

8

Database Design

13

9

UI/UX Design

14

10

Agent Orchestration Flow

15

11

Security & Compliance

16

12

Implementation Roadmap

17

13

Success Metrics & KPIs

18

14

Technology Stack

19

15

Risk Register & Mitigation

20

16

Budget & Resource Estimate

21

1. EXECUTIVE SUMMARY

AIR (Agentic Incident Response) is a next-generation, AI-driven incident management platform purpose-built for American Express India's complex, multi-system technology landscape. Drawing on proven patterns from industry leaders (ilert, PagerDuty, FireHydrant) and enriched with agentic AI capabilities powered by LLMs, Model Context Protocol (MCP), and Spring AI, AIR transforms how our engineering teams detect, diagnose, remediate, and learn from production incidents.

Purpose	Automate incident detection, triage, remediation and communication across all AmEx India platforms
Target Users	SRE Teams, Platform Engineers, Dev Teams, Engineering Leadership, NOC
Core Technology	Java Spring Boot, Spring AI, Python Agents, React Dashboard, PostgreSQL, Weaviate, Redis, MCP Servers
Autonomy Model	Progressive L1 (Co-Pilot) → L2 (Act-with-Approval) → L3 (Guardrailed Autonomy)
Primary Integration	Slack (primary), Datadog, PagerDuty, Kubernetes, GitHub, AWS/GCP, JIRA, ServiceNow
Projected MTTR Gain	60% reduction in Mean Time to Resolution within 12 months of full deployment
Cost Avoidance	Est. \$2.4M/year via reduced P1 downtime, engineer toil elimination, auto-resolution
Timeline	16-week phased rollout – Pilot (Wk 1-4) → MVP (Wk 5-10) → GA (Wk 11-16)

2. BUSINESS CASE & STRATEGIC VALUE

2.1 The Problem Worth Solving

American Express India processes millions of transactions daily across payments, rewards, servicing, and digital channels. Every minute of P1 downtime carries significant financial and reputational cost. Current incident response is largely manual, siloed, and reactive — consuming ~40% of senior engineering bandwidth.

~45 min

\$85K+

~60%

3 AM

Average P1 MTTR
(current state)
Cost per hour
of P1 downtime
Engineer time
on toil/repeat fixes
Avg wake-up time
for trivial restarts

2.2 Business Benefits

Operational Efficiency	Financial Impact
Auto-resolve 40–60% of P2/P3 incidents without human intervention	Est. \$2.4M annual savings: reduced downtime + toil elimination + faster resolution
Reduce L1 support tickets by 50% via intelligent Slack triage bot	Avoid 2+ P1 incidents per month = \$170K+ cost avoidance monthly
Eliminate manual runbook execution for known failure patterns	Reduce on-call burnout → lower attrition → lower recruiting cost
Free ~2,500 engineering hours/year for feature development	
Risk & Compliance	Strategic Advantage
Full audit trail of every AI action for RBI/regulatory compliance	Position AmEx India as an AI-first engineering organization
Consistent SLA adherence with automated escalation guardrails	Platform extensible to 20+ internal services and third-party tools
Postmortem automation ensures 100% incident documentation coverage	Reusable MCP server library for future AI agent initiatives

3. PROBLEM STATEMENT & CURRENT STATE

The current incident management approach at American Express India exhibits structural inefficiencies rooted in tool fragmentation, manual workflows, and lack of intelligent correlation. Key pain points:

Pain Point	Description	Severity
Alert Noise & Fatigue	Thousands of raw alerts daily; no deduplication or semantic grouping. Engineers ack >70% noise.	HIGH
Siloed Context	No unified view: metrics in Datadog, logs in Splunk, code in GitHub, tickets in JIRA — stitched manually.	HIGH
Human Bottlenecks	L1 → L2 → L3 escalation path averages 18 min before right expert engaged.	CRITICAL
Reactive Posture	Issues detected by customers or monitoring thresholds; no predictive/early-warning capability.	HIGH
Manual Runbooks	Known fixes (restart, scale, rollback) executed manually every time — no automation.	MEDIUM
Postmortem Gap	Only ~35% of P2+ incidents have documented postmortems; learnings not applied.	MEDIUM
On-Call Burnout	Engineers paged 3–5x/night for trivial issues; leading to alert fatigue and attrition.	HIGH

4. AIR PRODUCT VISION & SCOPE

4.1 Vision Statement

"AIR is the always-on, AI-native incident command centre for American Express India — autonomously detecting, diagnosing, resolving and learning from incidents end-to-end, while keeping humans in control of every high-stakes decision."

4.2 Core Product Capabilities

■ Intelligent Alerting	Semantic deduplication using vector embeddings; alert grouping; noise suppression; priority scoring via LLM.
■ AI-Driven Triage	LLM-powered root cause analysis with evidence from logs, metrics, deployments, and runbooks via RAG.
■ Automated Remediation	Pre-approved runbook execution (restart, rollback, scale) at L2/L3 with blast-radius checks and auto-rollback.
■ Slack-Native Workflow	Full incident lifecycle in Slack: alert, triage, approve actions, update status page, close — zero context switching.
■ AI Postmortems	Auto-generated postmortem docs with timeline, root cause, impact, action items from incident chat + logs + alerts.
■ Live Dashboard	React-based real-time dashboard: active incidents, MTTR trends, agent activity, autonomy metrics, SLA health.
■ Universal Integrations	MCP-based connectors for Datadog, Splunk, K8s, GitHub, JIRA, ServiceNow, AWS, GCP, PagerDuty and more.
■ ■ Human Oversight	Configurable guardrails, approval gates, circuit-breakers; full audit log; opt-out per team; no black-box decisions.

5. HIGH-LEVEL ARCHITECTURE

5.1 System Architecture Layers

AIR follows a layered, microservices-based architecture with a clear separation between data ingestion, AI reasoning, agent execution, tool integration, and persistence. All inter-service communication uses async event streaming (Kafka) and synchronous REST/gRPC where latency is critical.

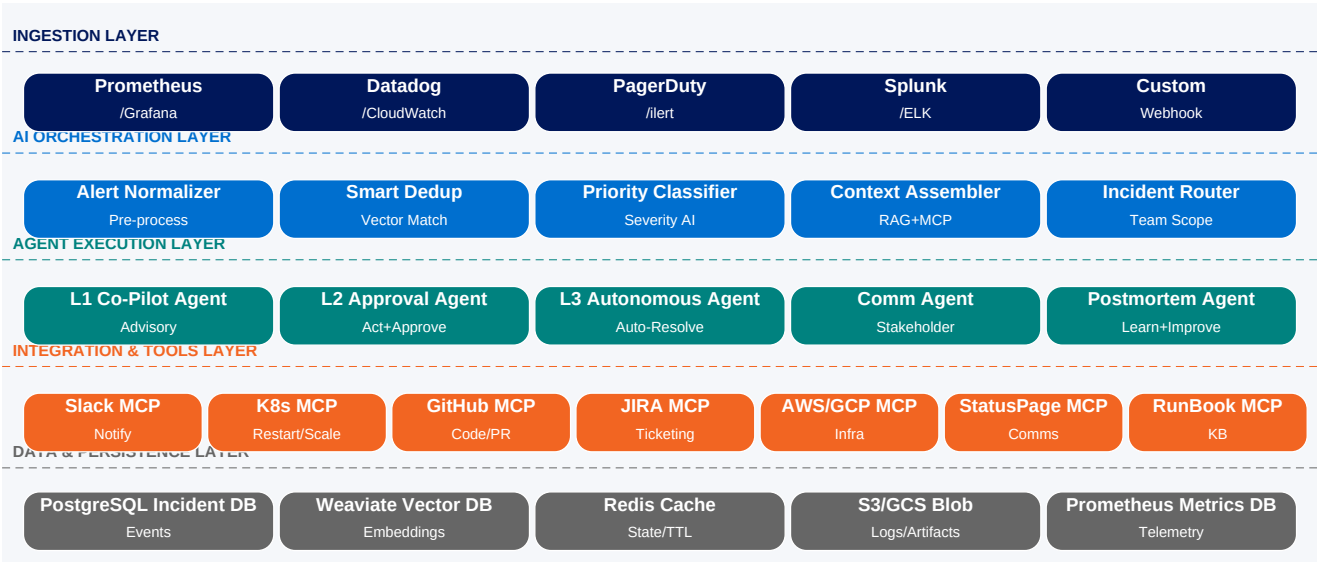


Figure 1: AIR End-to-End System Architecture (5-Layer Model)

5.2 Autonomy Levels – L1, L2, L3

AIR implements a progressive autonomy model adapted for enterprise safety requirements. Teams start at L1 and graduate to higher levels as trust in the AI is established through measured accuracy and reliability gates.

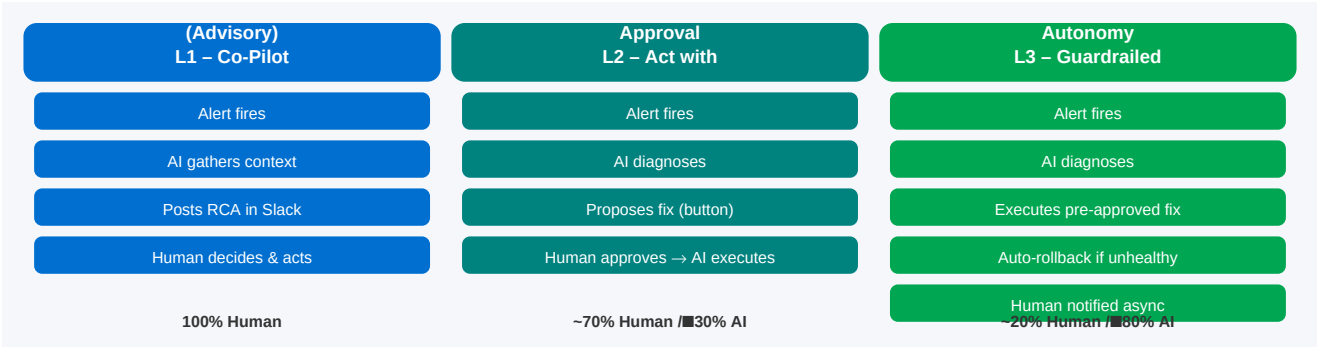


Figure 2: AIR Autonomy Progression Model – L1 Advisory → L2 Act-with-Approval → L3 Guardrailed Autonomy

6. LOW-LEVEL DESIGN

6.1 Alert Ingestion & Normalisation

Every incoming alert — regardless of source — is normalised into a canonical AlertEvent schema before entering the AI pipeline. This decouples alert sources from AI logic and enables cross-source correlation.

```
// AlertEvent Canonical Schema (Java)
@Data @Builder
public class AlertEvent {
    String alertId; // UUID
    String source; // "datadog" | "prometheus" | "pagerduty"
    String title; // Normalised title
    String description; // Plain text, no markdown
    Severity severity; // P1 | P2 | P3 | P4
    String serviceId; // Owning service identifier
    String teamId; // Owning team
    Map<String,String> labels; // k/v metadata
    Instant firedAt; // Original alert time
    List<String> affectedResources;
    float[] embedding; // 1536-dim vector (OpenAI ada-002)
}
```

6.2 AI Orchestration Engine

The Orchestration Engine is the central brain — a Spring AI-powered service that manages the ReAct (Reason + Act) loop for each incident. It selects the appropriate LLM, assembles context from RAG and MCP tools, and produces structured decisions.

Component	Technology	Responsibility
ContextAssembler	Java / Spring AI	Pulls logs, metrics, deployments via MCP into LLM context window. Uses RAG for runbook retrieval.
AlertDeduplicator	Python / Weaviate	Cosine similarity search on alert embeddings. Groups alerts with score > 0.88 threshold.
PriorityClassifier	Spring AI / GPT-4o	Classifies severity (P1-P4), identifies blast radius, estimates user impact.
IncidentRouter	Java / Rules + AI	Routes incident to correct team-scoped agent based on service ownership and ML classification.
ActionPlanner	Spring AI / ReAct	Generates step-by-step remediation plan; validates against runbook; checks preconditions.
GuardrailEngine	Java / Policy DSL	Enforces action whitelists, blast-radius limits, rollback requirements, approval gates.
ResponseFormatter	Spring AI / GPT-4o-mini	Generates human-readable Slack messages, status page updates, postmortem drafts.

6.3 Agent Execution Framework

Each team deploys one or more domain-scoped agents. Agents are stateful, long-running processes that maintain incident context across turns. They use Spring AI's tool-calling abstraction over MCP to interact with external systems.

Agent Type	Autonomy Level	Responsibilities
TriageAgent	L1 — Read Only	Gathers all context, produces RCA panel. No write access. Posts to Slack channel.
RemediationAgent	L2 — Approved Writes	Executes pre-approved actions after human approval via Slack button. Verifies success.
AutoResolveAgent	L3 — Guardrailed Auto	Executes allow-listed runbooks autonomously. Monitors outcome. Auto-rollback if needed.

CommunicationAgent	All Levels	Manages status page, Slack updates, stakeholder notifications throughout incident lifecycle.
PostmortemAgent	Post-Incident	Synthesises chat, alerts, actions into structured postmortem. Auto-creates JIRA action items.

6.4 MCP Tools & Function Registry

All external integrations are exposed as MCP (Model Context Protocol) servers, enabling AI agents to call tools with type-safe, audited function calls. Each tool call is logged with agent ID, timestamp, input, and output.

Tool / Function	MCP Server	Description	Access
slack_send_message	Slack MCP	Posts message/attachment to Slack channel or DM	Read/Write
slack_create_incident_channel	Slack MCP	Creates dedicated #inc-NNNN channel, invites team	Write
k8s_get_pod_status	K8s MCP	Returns pod status, restart count, OOMKill flags	Read
k8s_restart_deployment	K8s MCP	Restarts named deployment. Requires L2+ approval.	Write (Guarded)
k8s_scale_deployment	K8s MCP	Scales replica count. Requires L2+ approval.	Write (Guarded)
datadog_query_metrics	Datadog MCP	Fetches time-series metrics for given query & window	Read
splunk_search_logs	Splunk MCP	Executes SPL query, returns structured log results	Read
github_get_commits	GitHub MCP	Recent commits/PRs for service repo (change correlation)	Read
github_create_pr	GitHub MCP	Creates PR with AI-generated fix patch	Write (L3)
jira_create_ticket	JIRA MCP	Creates incident or action item ticket with full context	Write
statuspage_update	StatusPage MCP	Updates public/internal status page component status	Write
aws_describe_instances	AWS MCP	Returns EC2/ECS/RDS instance health and state	Read
runbook_search	KB MCP	Vector-similarity search over runbook knowledge base	Read
pagerduty_create_alert	PagerDuty MCP	Creates/escalates PagerDuty alert with severity and summary	Write

7. API SPECIFICATION

AIR exposes a RESTful API (JSON over HTTPS) and a Slack Events API webhook. All endpoints require Bearer JWT authentication. Rate limiting: 100 req/min per team. Base URL: <https://air.amex-internal.com/api/v1>

Method	Endpoint	Description	Response	Priority
POST	/alerts/ingest	Ingest normalised alert from any source	201 AlertEvent	P0
GET	/alerts	List alerts with filters (status, severity, team, window)	200 AlertList	P0
GET	/alerts/{id}	Get alert detail with embeddings, dedup group	200 AlertEvent	P0
PUT	/alerts/{id}/acknowledge	Acknowledge alert; optionally suppress	200 AlertEvent	P1
POST	/incidents	Create new incident from alert(s)	201 Incident	P0
GET	/incidents	List incidents (active/resolved, team, severity)	200 IncidentList	P0
GET	/incidents/{id}	Get full incident detail: timeline, agents, actions	200 Incident	P0
PATCH	/incidents/{id}	Update incident status, severity, assignee	200 Incident	P1
POST	/incidents/{id}/resolve	Mark resolved; triggers postmortem agent	200 Incident	P0
POST	/incidents/{id}/escalate	Manual escalation to next tier	200 Incident	P1
GET	/incidents/{id}/actions	Get list of proposed and executed agent actions	200 ActionList	P1
POST	/incidents/{id}/actions/{actionId}/approve	Human approval for L2 action	200 Action	P0
POST	/incidents/{id}/actions/{actionId}/reject	Reject proposed action with reason	200 Action	P0
GET	/incidents/{id}/rca	Get AI-generated root cause analysis	200 RCA	P1
GET	/incidents/{id}/postmortem	Get AI-generated postmortem document	200 Postmortem	P1
PUT	/incidents/{id}/postmortem	Update/finalise postmortem	200 Postmortem	P1
GET	/teams/{teamId}/agents	Get team's agent configurations	200 AgentConfig[]	P2
PUT	/teams/{teamId}/agents/{agentId}	Update agent autonomy level, guardrails	200 AgentConfig	P1
GET	/runbooks	List/search runbooks in knowledge base	200 RunbookList	P2
POST	/runbooks	Add runbook to knowledge base (vectorised)	201 Runbook	P2
GET	/metrics/mttr	MTTR breakdown by team, service, period	200 MTTRMetrics	P2
GET	/metrics/agents	Agent accuracy, acceptance rates, autonomy stats	200 AgentMetrics	P2
POST	/webhooks/slack/events	Receive Slack Event API callbacks	200 OK	P0
POST	/webhooks/pagerduty	Receive PagerDuty webhook events	200 OK	P0

8. DATABASE DESIGN

AIR uses PostgreSQL as the primary relational store, Weaviate for vector embeddings (alert deduplication, runbook RAG), and Redis for ephemeral agent state and rate limiting. All tables use UUID primary keys, soft deletes, and audit timestamps.

alerts	Core alert store	incidents	Incident lifecycle
id	UUID PK	id	UUID PK
source	VARCHAR(50)	title	TEXT
title	TEXT	severity	ENUM
description	TEXT	status	ENUM
severity	ENUM(P1-P4)	alert_ids	UUID[]
service_id	UUID FK	assignee_id	UUID FK
team_id	UUID FK	team_id	UUID FK
status	ENUM	service_ids	UUID[]
dedup_group_id	UUID	opened_at	TIMESTAMPTZ
fired_at	TIMESTAMPTZ	resolved_at	TIMESTAMPTZ
acknowledged_at	TIMESTAMPTZ	mttr_seconds	INT
created_at	TIMESTAMPTZ	autonomy_level	ENUM(L1/L2/L3)
labels	JSONB	slack_channel_id	VARCHAR

agent_actions	AI actions log	rca_results	AI root cause analysis
id	UUID PK	id	UUID PK
incident_id	UUID FK	incident_id	UUID FK
agent_type	VARCHAR(50)	agent_id	UUID
action_type	VARCHAR(100)	hypothesis	TEXT
status	ENUM(proposed/approved/executed/rolled_back)	supporting_evidence	JSONB
proposed_at	TIMESTAMPTZ	confidence_score	FLOAT
approved_by	UUID FK	recommended_actions	TEXT[]
approved_at	TIMESTAMPTZ	model_used	VARCHAR
executed_at	TIMESTAMPTZ	prompt_version	VARCHAR
input_payload	JSONB	generated_at	TIMESTAMPTZ
output_payload	JSONB	accepted_by	UUID
rollback_action_id	UUID FK	accepted_at	TIMESTAMPTZ
guardrail_checks	JSONB		

postmortems		Post-incident analysis		runbooks		Runbook knowledge base	
id	UUID	PK		id	UUID	PK	
incident_id	UUID	FK	UNIQUE	title	VARCHAR(200)		
status	ENUM(draft/review/published)			service_id	UUID	FK	
timeline	JSONB			team_id	UUID	FK	
impact	TEXT			content	TEXT		
root_cause	TEXT			embedding_id	VARCHAR		
contributing_factors	TEXT[]			tags	TEXT[]		
action_items	JSONB			version	INT		
generated_at	TIMESTAMPTZ			trigger_patterns	TEXT[]		
published_at	TIMESTAMPTZ			success_criteria	TEXT		
jira_epic_id	VARCHAR			last_used_at	TIMESTAMPTZ		
author_id	UUID			use_count	INT		
reviewed_by	UUID			created_at	TIMESTAMPTZ		

teams		Team registry		agent_telemetry		LLM observability	
id	UUID	PK		id	UUID	PK	
name	VARCHAR(100)			agent_type	VARCHAR		
slack_channel_id	VARCHAR			incident_id	UUID	FK	
on_call_schedule_id	VARCHAR			model_used	VARCHAR		
default_autonomy_level	ENUM			prompt_version	VARCHAR		
agent_config	JSONB			input_tokens	INT		
escalation_policy	JSONB			output_tokens	INT		
services	UUID[]			latency_ms	INT		
				feedback	ENUM(positive/negative/none)		
				error_type	VARCHAR		
				created_at	TIMESTAMPTZ		

9. UI/UX DESIGN

The AIR Dashboard is a React 18 + TypeScript SPA served at **air.amex-internal.com**. It uses the AmEx Design System tokens, Tailwind CSS, and Recharts for data visualisation. The interface is optimised for NOC screens (wide monitors) and mobile on-call use.

9.1 Key Views

Live Incident Feed	Real-time list of active incidents sorted by severity. Color-coded P1/P2/P3/P4. Shows AI confidence, autonomy level, time elapsed, assigned agent. One-click to open incident war room.
Incident War Room	Full incident timeline, AI RCA panel, evidence cards (logs, metrics, commits). Approve/Reject action buttons for L2 actions. Live Slack feed embed. Postmortem trigger.
Agent Activity Panel	Shows each agent's current reasoning step, tool calls made, tokens used, confidence score. Expandable trace view. Feedback thumbs up/down. Model and prompt version visible.
Alert Explorer	Grouped alert view with dedup clusters. Similarity score badges. Filter by source, service, severity, time. Bulk acknowledge. Noise ratio analytics.
Runbook Manager	Browse, create, edit runbooks. Shows usage frequency, last triggered date, associated services, trigger patterns. Test against historical incidents.
Metrics Dashboard	MTTR trends (line chart), auto-resolution rate (gauge), alert volume (bar), agent accuracy heatmap, SLA compliance gauge, on-call load by engineer.
Team Config	Configure autonomy level (L1/L2/L3), agent guardrails, action whitelist, escalation policy, Slack channel binding, on-call schedule integration.

9.2 Slack Experience Design

The Slack integration is the primary interface for on-call engineers. AIR uses Slack Block Kit for rich, interactive messages with approve/reject buttons, expandable evidence sections, and inline status updates.

[illegible]

Figure 3: Sample AIR Slack Action Card (L2 Approval Flow)

10. AGENT ORCHESTRATION FLOW

The following describes the complete ReAct agent loop from alert ingestion to resolution. The orchestration engine uses Spring AI's ChatClient with tool-calling and a custom state machine to track incident progress.

1. Alert Ingested

- Alert arrives via webhook (Datadog, Prometheus, PagerDuty)
- AlertNormalizer maps to canonical AlertEvent schema
- Embedding generated via OpenAI text-embedding-ada-002
- DedupService queries Weaviate — similarity search (cosine > 0.88)
- If duplicate: attach to existing alert group → suppress paging

2. Incident Created

- New incident record created in PostgreSQL
- Incident severity scored by PriorityClassifier (LLM)
- Team-scoped agent activated based on service ownership
- Slack channel #inc-{NNNN} created, team + on-call invited
- On-call human notified (parallel to AI triage)

3. AI Triage (ReAct Loop)

- ContextAssembler pulls: recent logs (Splunk), metrics (Datadog), last 5 commits (GitHub)
- LLM reasons: "What changed? What failed? What does the runbook say?"
- RCA generated with evidence links and confidence score
- Recommended actions ranked by safety (reversibility, blast radius)
- All findings posted to Slack war room channel as structured card

4. Action Execution (L1/L2/L3)

- L1: Human reads RCA, decides, acts manually. AI provides commands.
- L2: AI proposes action → Slack button → Human approves → Agent executes via MCP tool
- L3: Agent checks action against whitelist + guardrail engine → executes → monitors outcome
- All actions logged in agent_actions table with full input/output payload
- Auto-rollback triggered if health check fails within 3 minutes

5. Resolution & Learning

- Incident resolved: MTTR recorded, SLA checked
- PostmortemAgent activated: synthesises timeline, RCA, impact, action items
- Postmortem draft posted to Slack, JIRA epic created with action items
- Agent telemetry written: tokens, latency, feedback, model version
- Runbook knowledge base updated with new learnings (if applicable)

11. SECURITY & COMPLIANCE

Security Control	Implementation Detail
Authentication	OAuth 2.0 / SSO via AmEx Identity Provider. Slack OAuth for bot identity. Service-to-service mTLS.
Authorisation	RBAC: roles = viewer / responder / approver / admin. Team-scoped data isolation. Agent permissions per action type.
Data Privacy	No PII/CHD in AI prompts. Customer data masked at ingestion. Alert text sanitised before LLM calls.
LLM Data Handling	Azure OpenAI Service (EU data centres). Data NOT used for model training. Opt-out available per team.
Agent Safety	All destructive actions require L2 approval or L3 whitelist. Non-reversible actions require explicit human approval always.
Audit Logging	All API calls, agent actions, LLM interactions logged to immutable audit store. 90-day retention. SIEM integration.
Secret Management	All API keys via HashiCorp Vault. No secrets in code or environment variables. Rotation automated.
Network Security	MCP servers in private VPC subnet. Firewall rules: agents can only reach pre-approved endpoints.
Compliance	Aligned with RBI IT Framework, PCI-DSS (no card data in scope), SOC 2 Type II principles, ISO 27001.
Prompt Injection	System prompts are static and hardened. User/alert content never interpreted as instructions. Output validated by schema.

12. IMPLEMENTATION ROADMAP

16-week phased rollout across 4 phases, targeting production readiness by Week 16. Each phase has clear entry/exit criteria and a dedicated rollback plan.

Phase 1: Foundation & Pilot

- Core Spring Boot microservices skeleton (alert-ingest, incident-mgmt, agent-service)
- Alert normalisation pipeline + Kafka event bus setup
- Alert deduplication with Weaviate vector store (pilot with Datadog alerts)
- Basic Slack bot integration (alert notifications, manual acknowledge)
- PostgreSQL schema + Flyway migrations for core tables
- L1 TriageAgent with Spring AI ChatClient + Datadog MCP (read-only)
- React dashboard MVP: incident list, alert explorer
- Pilot with 1 service team (Payments SRE)

Phase 2: AI Core & Integrations

- Full RAG pipeline: runbook ingestion → Weaviate → ContextAssembler
- PriorityClassifier and RCA generation with evidence cards
- Slack interactive messages: Block Kit action cards (L2 approve/reject)
- K8s MCP, GitHub MCP, Splunk MCP integrations (read access)
- L2 RemediationAgent with approval workflow (restart, rollback)
- GuardrailEngine with action whitelist and blast-radius checks
- JIRA MCP for ticket creation, StatusPage MCP
- Expand to 3 service teams, 25+ runbooks loaded

Phase 3: Automation & Postmortems

- L3 AutoResolveAgent with pre-approved action execution
- PostmortemAgent with auto-generated structured documents
- Agent telemetry dashboard (LLM observability, feedback loops)
- AWS/GCP MCP for infrastructure actions (scale, restart)
- On-call schedule integration (PagerDuty API)
- Auto-rollback mechanism with health check polling
- Full Metrics Dashboard (MTTR, resolution rate, SLA compliance)
- Expand to all SRE teams, integration regression test suite

Phase 4: Scale & Production Hardening

- Performance tuning: sub-5s triage response at P99
- Multi-model strategy: GPT-4o for complex RCA, GPT-4o-mini for formatting
- HA deployment on Kubernetes (3 replicas, HPA, PodDisruptionBudgets)
- Redis cluster for agent state with TTL-based cleanup
- Full security audit + penetration testing

- Runbook A/B testing framework (promptfoo integration)
- Onboard all AmEx India engineering teams (15+ teams)
- GA launch + Director/VP demo, success metrics review

13. SUCCESS METRICS & KPIS

KPI	Baseline	Phase 2 Target	Phase 4 Target	Cadence
Mean Time to Resolution (MTTR)	45 min (P1)	< 18 min (P1)	< 15 min	Monthly
Auto-Resolution Rate	0%	20% (P3/P4)	50% (P2/P3/P4)	Weekly
Alert Noise Ratio	~70% noise	< 40% noise	< 20% noise	Weekly
AI Suggestion Acceptance	N/A	> 60%	> 80%	Per sprint
Postmortem Completion	35%	90%	100%	Monthly
P1 Incidents < 30 min	~30%	> 60%	> 80%	Monthly
On-call Pages (trivial)	Baseline	–30%	–60%	Monthly
Engineer Hours Saved/Month	Baseline	200 hrs	400+ hrs	Monthly
Agent Telemetry Feedback	N/A	> 70% positive	> 85% positive	Weekly
SLA Compliance	Baseline	+10%	+20%	Monthly

14. TECHNOLOGY STACK

Layer / Category	Technology Choices
Backend – Core Services	Java 21 + Spring Boot 3.3, Spring AI 1.0 GA, Spring Data JPA, Spring Kafka, Spring Security OAuth2
AI / LLM	OpenAI GPT-4o / GPT-4o-mini via Azure OpenAI Service; text-embedding-ada-002 for vectors; promptfoo for testing
Agent Framework	Spring AI ChatClient + Tool Calling; Custom ReAct loop state machine; LangChain4j (optional)
MCP Integration	Model Context Protocol (MCP) over SSE/HTTP; Custom MCP server stubs in Java + Python
Vector Database	Weaviate 1.24 (self-hosted on K8s); cosine similarity; schema per alert class
Primary Database	PostgreSQL 16 (RDS/Cloud SQL); Flyway migrations; connection pooling via HikariCP
Cache / State	Redis 7.2 Cluster; Jedis/Lettuce client; TTL-based agent state; distributed locks
Message Bus	Apache Kafka 3.6; topics: raw-alerts, normalised-alerts, incidents, agent-actions, telemetry
Frontend	React 18 + TypeScript; Vite; Tailwind CSS; Recharts; React Query; Zustand; Slack Block Kit
Infrastructure	Kubernetes 1.30 (EKS/GKE); Helm charts; ArgoCD GitOps; Horizontal Pod Autoscaler
Observability	Prometheus + Grafana; Jaeger distributed tracing; ELK stack for AIR logs; custom LLM telemetry DB
CI/CD	GitHub Actions; SonarQube; JUnit 5 + Mockito; Testcontainers; promptfoo; ArgoCD
Security	HashiCorp Vault; mTLS (Istio); OWASP dependency check; Trivy container scanning
Slack Integration	Slack Bolt SDK (Java); Block Kit; Events API; Socket Mode for dev; Webhook for prod

15. RISK REGISTER & MITIGATION

ID	Risk	Likelihood	Impact	Mitigation Strategy
R1	AI Hallucination on RCA	HIGH	MEDIUM	RAG grounding; evidence requirements; confidence threshold gates; human L1 fallback always available
R2	L3 Agent Wrong Action	MEDIUM	HIGH	Strict action whitelist; blast-radius checks; auto-rollback within 3 min; human notification always sent
R3	Prompt Injection via Alerts	MEDIUM	HIGH	Static system prompt; alert text never parsed as instructions; output schema validation
R4	LLM API Outage (Azure OpenAI)	LOW	HIGH	Fallback to smaller local model (Ollama); graceful degrade to L1 manual mode; circuit breaker
R5	Data Privacy Breach	LOW	CRITICAL	PII/CHD masking at ingestion; EU data residency; Azure OpenAI no-training contract; DSPM controls
R6	Engineer Over-Reliance on AI	MEDIUM	MEDIUM	Training programme; L1 mode default for new teams; transparency in AI reasoning shown always
R7	Alert Volume Spike (LLM Cost)	MEDIUM	LOW	Dedup reduces volume; GPT-4o-mini for bulk tasks; token budget per incident; cost dashboard
R8	Integration Downtime (MCP servers)	MEDIUM	MEDIUM	Graceful degradation; AI clearly states missing context; tools have mock fallback for read ops

16. BUDGET & RESOURCE ESTIMATE

16.1 Team Composition (16 Weeks)

Role	Count	Duration	Responsibilities
Staff Architect / Tech Lead	1	16 weeks	Architecture, design, cross-cutting decisions
Senior Backend Engineers (Java/Spring)	3	16 weeks	Core services, agent framework, MCP servers
AI/ML Engineer (Python/LLMs)	1	16 weeks	Agent logic, RAG pipeline, vector store, LLM ops
Frontend Engineer (React)	1	12 weeks	Dashboard, Slack Block Kit UX
DevOps / Platform Engineer	1	16 weeks	K8s, CI/CD, infrastructure, observability
QA / Test Engineer	1	12 weeks	Integration tests, LLM eval (promptfoo), security testing
Product Manager	1	16 weeks	Roadmap, stakeholder alignment, success metrics

16.2 Infrastructure & Tooling Cost (Annual)

Cost Item	Est. Annual Cost	Notes
Azure OpenAI Service (GPT-4o + embeddings)	~\$45,000/yr	Based on 500K incidents/yr at avg 5K tokens each
Kubernetes Cluster (EKS/GKE) for AIR services	~\$28,000/yr	8-node cluster, HA, staging + prod
Weaviate Cloud / Self-hosted	~\$8,000/yr	Vector store for 10M+ embeddings
PostgreSQL RDS (Multi-AZ)	~\$9,600/yr	db.r6g.large, 500GB, automated backups
Redis ElastiCache Cluster	~\$6,000/yr	3-node r6g.large cluster
Kafka MSK / Confluent Cloud	~\$12,000/yr	3-broker cluster, 7-day retention
Monitoring (Grafana Cloud)	~\$4,800/yr	Metrics, logs, traces for AIR platform
Slack Premium API / Enterprise	Existing	No incremental cost (existing contract)
TOTAL (Infrastructure)	~\$113,400/yr	

16.3 ROI Summary

Benefit / Cost	Estimate	Basis
P1 Downtime Reduction (60% MTTR cut)	\$1,200,000	Based on 12 P1s/month × \$85K/hr × 60% improvement
Engineer Toil Elimination (400 hrs/month)	\$720,000	Based on avg \$150/hr fully-loaded senior engineer cost
Reduced On-Call Burnout / Attrition	\$480,000	Est. 2 senior engineer replacements avoided at \$240K each
TOTAL PROJECTED ANNUAL SAVINGS	\$2,400,000	
TOTAL INVESTMENT (Infra + 16-wk build)	~\$600,000	Infra \$113K + ~\$487K engineering (8 FTEs × 16 weeks)
NET FIRST-YEAR BENEFIT	\$1,800,000	3x ROI in Year 1; 20x+ in Year 2+

Next Steps: This document is submitted for Director and VP review and budget approval. Upon approval, the Architecture team will proceed with Phase 1 kickoff within 2 weeks, beginning with Payments SRE team pilot. A detailed sprint-by-sprint execution plan and stakeholder update cadence will be shared separately.

Prepared by: Enterprise Architecture Team, American Express India | Date: June 2025 | Version: 1.0